

FORTIGATE NSE4

Lab 9 — Site-to-Site IPsec VPN

1. IPsec Fundamentals

Two protocol families cover the two kinds of VPN. IPsec is used for site-to-site tunnels between fixed gateways; SSL-VPN is used for remote access, where an individual user connects from an unpredictable address.

► 1.1 What IPsec Provides

Benefit	What it means
Confidentiality	The payload is encrypted, so an observer on the transit path cannot read it
Integrity	Any modification in transit is detected through a hash over the packet
Authentication	Each peer proves its identity before any data is carried
Anti-replay protection	Sequence numbers ensure a captured packet cannot be re-sent later and accepted a second time
Key management	Keys are negotiated and periodically replaced without manual intervention

Each benefit has its own mechanism. IPsec is not one protocol but a family of them working together, which is why a tunnel can fail at several independent points.

► 1.2 The Component Protocols

Protocol	Role
IKE	Internet Key Exchange — negotiates the tunnel and manages keys. Runs on UDP 500, or UDP 4500 when NAT traversal is in use
ESP	Encapsulating Security Payload — provides encryption and integrity for the carried traffic
AH	Authentication Header — provides integrity and authentication only, with no encryption

► 1.3 IKE Phases

IKE establishes the tunnel in two stages. Phase 1 builds a secure channel between the two gateways; phase 2 uses that channel to negotiate the keys that protect user traffic.

Phase 1 performs three things in sequence: negotiation of the proposal set, a Diffie-Hellman key exchange, and authentication of each peer.

Mode	Behaviour
Main mode	Six messages. Identity is exchanged after encryption is established, so it is not visible to an observer
Aggressive mode	Three messages. Faster, but identity and a hash of the pre-shared key are sent before encryption is in place, which exposes them to a passive observer

► 1.4 Transport and Tunnel Mode

Mode	What is encrypted	Used for
Transport	The payload only — the original IP header is left in the clear	Host-to-host protection where the endpoints are the peers themselves
Tunnel	The entire original packet, including its IP header, wrapped in a new one	Site-to-site VPN. This is the mode used in this lab

► 1.5 The IPsec Lifecycle

A tunnel moves through four stages: initialisation, IKE negotiation, data transfer, and termination. Each of the tests later in this document targets one of them.

► 1.6 Two Related Concepts

Auto Discovery VPN (ADVPN). Allows spokes to discover one another and build direct shortcut tunnels on demand, so a hub-and-spoke design can behave as a full mesh without configuring every pair manually. It relies on a dynamic routing protocol to distribute reachability.

Dial-up VPN. Used when only one side has a predictable address. The server end must hold a static IP; the client end may connect from anywhere, which is why the server cannot identify it by address and must rely on credentials or certificates instead.

2. Topology and Addressing

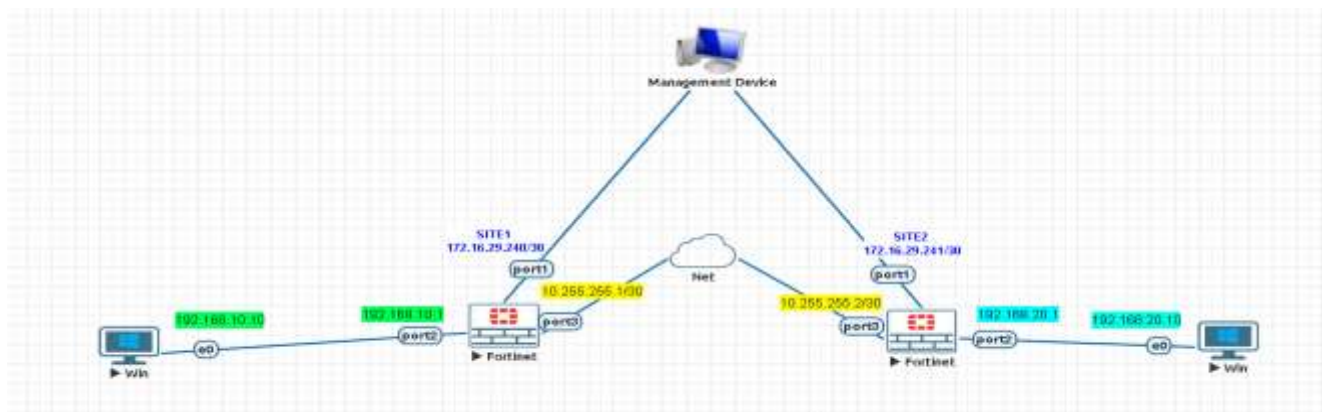


Figure 1 — Lab topology — two FortiGates joined by an isolated transit segment

Element	SITE1	SITE2
LAN interface (port2)	192.168.10.1/24	192.168.20.1/24
Client	192.168.10.10	192.168.20.10
VPN interface (port3)	10.255.255.1/30	10.255.255.2/30
Management (port1)	Cloud0	Cloud0

The transit segment is isolated. The two WAN interfaces share a private /30 on an EVE bridge with no route to anything else, so the peers reach each other directly and no default route is required.

3. Interface Configuration

▶ 3.1 LAN Interfaces

Edit Interface

Name:

Alias:

Type: ☒ Physical Interface

VRF ID:

Role:

☐ Dedicated Management Port

Address

Addressing mode: ☒ Manual ☐ DHCP ☐ Auto-managed by IPAM ☐ One-Arm Sniffer

IP/Netmask:

Create address object matching subnet: ☒

Name:

Destination:

Secondary IP address: ☐

Administrative Access

IPv4: ☐ HTTPS ☒ PING ☐ FMG-Access

☐ SSH ☐ SNMP ☐ FTM

Figure 2 — SITE1 port2 — LAN role, 192.168.10.1/24

Edit Interface

Name:

Alias:

Type: ☒ Physical Interface

VRF ID:

Role:

☐ Dedicated Management Port

Address

Addressing mode: ☒ Manual ☐ DHCP ☐ Auto-managed by IPAM ☐ One-Arm Sniffer

IP/Netmask:

Create address object matching subnet: ☐

Secondary IP address: ☐

Administrative Access

IPv4: ☐ HTTPS ☒ PING ☐ FMG-Access

☐ SSH ☐ SNMP ☐ FTM

☐ RADIUS Accounting ☐ Security Fabric Connection ☐ Speed Test

Figure 3 — SITE2 port2 — LAN role, 192.168.20.1/24

Both LAN interfaces have **Create address object matching subnet** enabled, which generates the firewall address objects the VPN wizard will reference later.

▶ 3.2 VPN Interfaces

Figure 4 — SITE1 port3 — WAN role, 10.255.255.1/30

Figure 5 — SITE2 port3 — WAN role, 10.255.255.2/30

Administrative access on both is limited to PING, which is all the underlay verification requires.

4. Underlay Verification

This step comes before any VPN configuration. **If the two peers cannot reach each other, every IKE error observed afterwards is misleading** — the symptoms point at the tunnel while the fault is one layer below it.

```

Site1 # execute ping 10.255.255.2
PING 10.255.255.2 (10.255.255.2): 56 data bytes
64 bytes from 10.255.255.2: icmp_seq=0 ttl=255 time=4.8 ms
64 bytes from 10.255.255.2: icmp_seq=1 ttl=255 time=0.7 ms
64 bytes from 10.255.255.2: icmp_seq=2 ttl=255 time=1.3 ms
64 bytes from 10.255.255.2: icmp_seq=3 ttl=255 time=1.7 ms
64 bytes from 10.255.255.2: icmp_seq=4 ttl=255 time=0.5 ms

--- 10.255.255.2 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 0.5/1.8/4.8 ms

```

Figure 6 — SITE1 reaching the SITE2 VPN interface

```

Site2 # execute ping 10.255.255.1
PING 10.255.255.1 (10.255.255.1): 56 data bytes
64 bytes from 10.255.255.1: icmp_seq=0 ttl=255 time=2.5 ms
64 bytes from 10.255.255.1: icmp_seq=1 ttl=255 time=1.5 ms
64 bytes from 10.255.255.1: icmp_seq=2 ttl=255 time=1.1 ms
64 bytes from 10.255.255.1: icmp_seq=3 ttl=255 time=1.3 ms
64 bytes from 10.255.255.1: icmp_seq=4 ttl=255 time=1.3 ms

--- 10.255.255.1 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 1.1/1.5/2.5 ms

```

Figure 7 — SITE2 reaching the SITE1 VPN interface

Each client was then tested against its own gateway, confirming the LAN side before the tunnel is introduced.

```

C:\Users\user>ping 192.168.10.1

Pinging 192.168.10.1 with 32 bytes of data:
Reply from 192.168.10.1: bytes=32 time<1ms TTL=255
Reply from 192.168.10.1: bytes=32 time<1ms TTL=255
Reply from 192.168.10.1: bytes=32 time<1ms TTL=255
Reply from 192.168.10.1: bytes=32 time=1ms TTL=255

Ping statistics for 192.168.10.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

```

Figure 8 — SITE1 client reaching its gateway

```

C:\Users\user>ping 192.168.20.1

Pinging 192.168.20.1 with 32 bytes of data:
Reply from 192.168.20.1: bytes=32 time<1ms TTL=255
Reply from 192.168.20.1: bytes=32 time=1ms TTL=255
Reply from 192.168.20.1: bytes=32 time=1ms TTL=255
Reply from 192.168.20.1: bytes=32 time=1ms TTL=255

Ping statistics for 192.168.20.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

```

Figure 9 — SITE2 client reaching its gateway

All interfaces are now confirmed working independently. Any failure after this point belongs to the VPN configuration.

5. Building the Tunnel — SITE1

The VPN Creation Wizard builds phase 1, phase 2, the address groups, the routes, and both firewall policies from a single set of inputs.



Figure 10 — Step 1 — Site to Site, FortiGate peer, no NAT between sites



Figure 11 — Step 2 — remote peer address and pre-shared key



Figure 12 — Step 3 — local and remote subnets, Internet Access set to None

Internet Access is set to None because this tunnel carries only site-to-site traffic. The other options would route one site's internet traffic through the other, which is a different design.



Figure 13 — The tunnel reports Inactive immediately after creation

This is expected rather than a fault. **Without auto-negotiate, phase 1 only starts when traffic matches the phase 2 selectors** — a tunnel with nothing to carry stays down.

6. Building the Tunnel — SITE2



Figure 14 — Step 1 — same template, mirrored



Figure 15 — Step 2 — peer address 10.255.255.1, same pre-shared key



Figure 16 — Step 3 — subnets reversed relative to SITE1



Figure 17 — SITE2 also reports Inactive

The subnets must be mirrored exactly. SITE1's local is SITE2's remote and vice versa. A mismatch here produces a tunnel that establishes phase 1 and then fails phase 2, which is a confusing symptom because the gateway appears to come up.

7. What the Wizard Created

Destination	Gateway IP	Interface	Status	Comments
to-Site2_remote		to-Site2	Enabled	VPN to-Site2 (Created by VPN wizard)
to-Site2_remote		Blackhole	Enabled	VPN to-Site2 (Created by VPN wizard)

Figure 18 — Routes created automatically — a static route and a blackhole route

Route	Purpose
Static route	Directs the remote subnet into the tunnel interface
Blackhole route	A higher-distance route to the same subnet pointing at null. If the tunnel drops, traffic for the remote LAN is discarded rather than following a default route out in the clear

The blackhole route is a genuine protection and one a manual build usually omits. Without it, a tunnel failure silently converts encrypted site-to-site traffic into plaintext internet traffic.



Figure 19 — Both firewall policies created — local to remote and remote to local

Two policies are required because **FortiOS policies are unidirectional**. Return traffic on an established session is handled by the session table, but a session initiated from the far side needs its own policy.

```
#show vpn ipsec phase1-interface to-Site2
```

```
show vpn ipsec phase2-interface to-Site2
```

```

Bitel # show vpn ipsec phase1-interface to-Site2
config vpn ipsec phase1-interface
  edit "to-Site2"
    set interface "port3"
    set peer-type any
    set nat-device disable
    set proposal des-md5 des-sha1
    set comments "VPN: to-Site2 (Created by VPN wizard)"
    set wizard-type static-fortigate
    set remote-gw 10.255.255.2
    set psksecret ENC 2FvHPI0EgMa/KV8g++2DIE/n0Sx2V8tKnjpsCape0Jr+mk405a67fC2XVq2af3AbWwA308x027906HgASuXr50cNs/HfH/9Hs0Ldn
    u09v4R8hQ22nd+TQh5o3XxYMLA079d3J0gc7scuLJ3a5x365cyR6oI897Bd5nd3I9a9s1Koc1gg7KtFuIw==
    set
  next
end

Bitel # show vpn ipsec phase2-interface to-Site2
config vpn ipsec phase2-interface
  edit "to-Site2"
    set phaseName "to-Site2"
    set proposal des-md5 des-sha1
    set comments "VPN: to-Site2 (Created by VPN wizard)"
    set src-addr-type name
    set dst-addr-type name
    set src-name "to-Site2-local"
    set dst-name "to-Site2-remote"
  next
end

```

Figure 20 — The generated phase 1 configuration viewed from the CLI

8. Tunnel Established

Generating traffic from the SITE1 client brought the tunnel up.



Figure 21 — The tunnel reporting Up

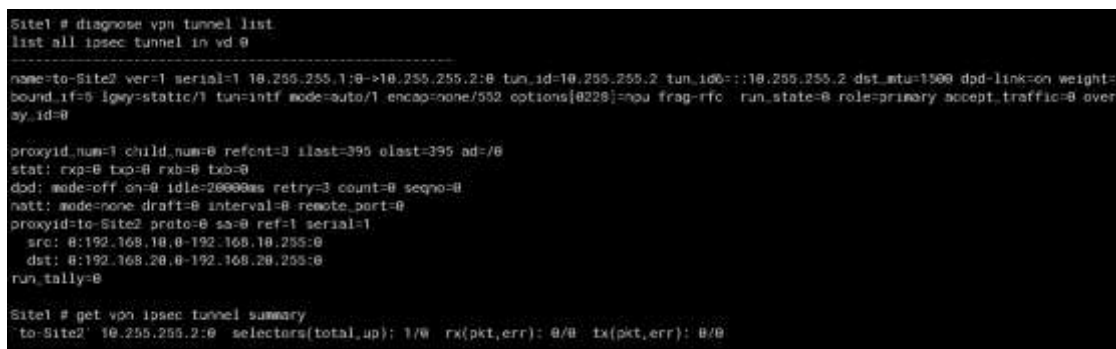


Figure 22 — Tunnel detail and summary — SA established, counters incrementing

The enc and dec counters confirm the tunnel is carrying traffic rather than merely being established.

9. Test 1 — Dead Peer Detection

DPD is a liveness mechanism. Without it, a gateway has no way to learn that its peer has disappeared, because IPsec is connectionless and a silent peer is indistinguishable from an idle one.

► 9.1 Part A — DPD Disabled

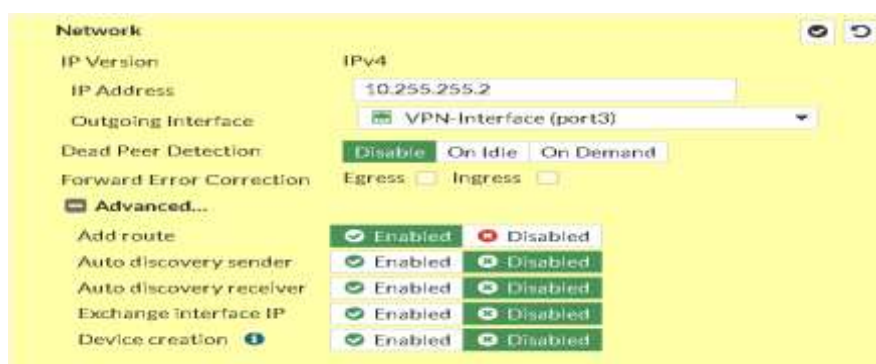


Figure 23 — DPD disabled on SITE2

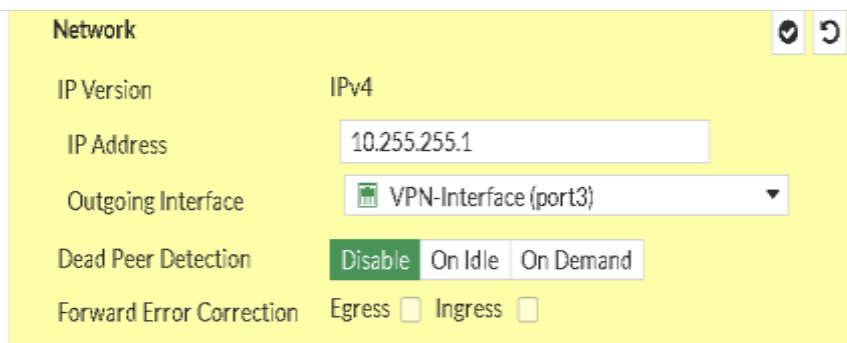


Figure 24 — DPD disabled on SITE1

The peer must be hard-killed, not shut down cleanly. A clean interface shutdown sends an IKE delete notification and the tunnel drops instantly — which bypasses the mechanism entirely and proves nothing. The SITE2 VM was stopped in EVE instead.

```
Site1 # diagnose vpn tunnel list
list all ipsec tunnel in vd 0

name-to-Site2 ver=1 serial=1 10.255.255.1:0->10.255.255.2:0 tun_id=10.255.255.2 tun_id0::10.255.255.2 dst_mtu=1500 dpd-link=on weight=1
bound_if=0 igmp=static/1 tun=ntf mode=auto/1 encap=none/532 options[0220]=npu frag-rfc run_state=0 role=primary accept_traffic=0 overl
ay_id=0

proxyid_num=1 child_num=0 reftot=3 ilast=395 elast=395 ad=/0
stat: rxp=0 txp=0 rxb=0 txb=0
dpd: mode=off on=0 idle=20000ms retry=3 count=0 seqno=0
natt: mode=none draft=0 interval=0 remote_port=0
proxyid-to-Site2 proto=0 sa=0 ref=1 serial=1
src: 0:192.168.10.0-192.168.10.255:0
dst: 0:192.168.20.0-192.168.20.255:0
run_tally=0

Site1 # get vpn ipsec tunnel summary
to-Site2' 10.255.255.2:0 selectors(total,up): 1/0 rx(pkt,err): 0/0 tx(pkt,err): 0/0
```

Figure 25 — SITE1 still reports the SA as up after the peer has gone

```
C:\Users\user>tracert -d 192.168.20.10

Tracing route to 192.168.20.10 over a maximum of 30 hops

 1      1 ms      <1 ms      <1 ms  192.168.10.1
 2      *        *        *      Request timed out.
 3      *        *        *      Request timed out.
 4      *        *        *      Request timed out.
```

Figure 26 — Traceroute from the SITE1 client returns nothing

This is the finding. The control plane reports a healthy tunnel while the data plane is dead. **SITE1 continues encrypting into a peer that no longer exists, and will do so until the phase 1 lifetime expires — up to 86400 seconds by default.**

► 9.2 Part B — DPD Enabled

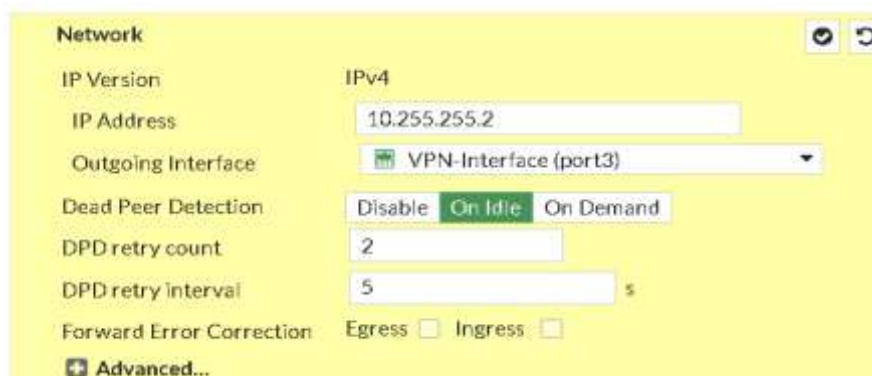


Figure 27 — DPD set to On Idle on SITE2 — retry count 2, interval 5 s



Figure 28 — DPD set to On Idle on SITE1 — retry count 3, interval 5 s

Mode	Behaviour
Disable	No liveness checking at all
On Demand	Probes only when traffic is queued but going unanswered — the default
On Idle	Probes continuously, even with no user traffic, so a dead peer is detected before anyone tries to use the tunnel

With retry interval 5 and retry count 2 or 3, the tunnel is declared dead within roughly **10 to 15 seconds** of the peer disappearing, rather than persisting for the full phase 1 lifetime.

10. Test 2 — Perfect Forward Secrecy

PFS forces a fresh Diffie-Hellman exchange each time the phase 2 keys are renewed, so the new keys are not derived from existing phase 1 material.

What it protects against: **without PFS, an attacker who recovers the phase 1 key can derive every phase 2 key that follows from it** — including for traffic captured earlier and stored. With PFS, each rekey is independent, so a single compromised key exposes only the traffic in that one interval.

► 10.1 Shortening the Key Lifetime

The phase 2 lifetime was reduced on both sites so a rekey occurs within the observation window rather than after the default interval.

```
Site1 # config vpn ipsec phase2-interface
Site1 (phase2-interface) # edit "to-Site2"
Site1 (to-Site2) # set keylifeseconds 300
```

Figure 29 — SITE1 phase 2 key lifetime set to 300 seconds

```
Site2 # config vpn ipsec phase2-interface
Site2 (phase2-interface) # edit "to-Site1"
Site2 (to-Site1) # set keylifeseconds 300
```

Figure 30 — SITE2 phase 2 key lifetime set to 300 seconds

► 10.2 Observing the Rekey

```
Site1 # diagnose debug reset
Site1 # diagnose debug application ike -1
Debug messages will be on for 30 minutes.
Site1 # diagnose debug enable

Site1 # ike 0:to-Site2: link is idle 5 10.255.255.1->10.255.255.2:0 dpd=1 seqno=48 rr=0
ike 0:to-Site2:21: send IKEv1 DPD probe, seqno 64
ike 0:to-Site2:21: enc 7528b194111cc1eA9933DA2917FE17A888100501377355AD00000050030000148A9D6F269D826A6f04A21258C531AC770000002000000010
1108D287528B194111CC1EAB933DA2917FE17A888100501377355AD00000050030000148A9D6F269D826A6f04A21258C531AC770000002000000010
ike 0:to-Site2:21: out 7528b194111cc1eA9933DA2917FE17A888100501377355AD00000050030000148A9D6F269D826A6f04A21258C531AC770000002000000010
25EEAC709E072E268C38D26653137E89FA6E7C3ADF2CBE4F889C555
ike 0:to-Site2:21: sent IKE msg (R-U-THERE): 10.255.255.1:500->10.255.255.2:500, len=84, vrf=0, id=7528b194111cc1eA9933DA2917FE17A888100501377
355ad
ike 0: comes 10.255.255.2:500->10.255.255.1:500, ifindex=5, vrf=0, ...
ike 0: IKEv1 exchange-Informational id=7528b194111cc1eA9933DA2917FE17A888100501377355AD00000050030000148A9D6F269D826A6f04A21258C531AC770000002000000010
ike 0: in 7528b194111cc1eA9933DA2917FE17A888100501377355AD00000050030000148A9D6F269D826A6f04A21258C531AC770000002000000010
85C9C795F36DA7441E1BAE96EA3AD755BC8CF255B
ike 0:to-Site2:21: dec 7528b194111cc1eA9933DA2917FE17A888100501377355AD00000050030000148A9D6F269D826A6f04A21258C531AC770000002000000010
1108D287528B194111CC1EAB933DA2917FE17A888100501377355AD00000050030000148A9D6F269D826A6f04A21258C531AC770000002000000010
ike 0:to-Site2:21: notify msg received: R-U-THERE
ike 0:to-Site2:21: enc 7528b194111cc1eA9933DA2917FE17A888100501377355AD00000050030000148A9D6F269D826A6f04A21258C531AC770000002000000010
1108D287528B194111CC1EAB933DA2917FE17A888100501377355AD00000050030000148A9D6F269D826A6f04A21258C531AC770000002000000010
ike 0:to-Site2:21: out 7528b194111cc1eA9933DA2917FE17A888100501377355AD00000050030000148A9D6F269D826A6f04A21258C531AC770000002000000010
363DA16C1BE547FAD8C81088EF8533B06FB5D76F4EAC4898C9C08DA
```

Figure 31 — IKE debug output showing the rekey exchange

```
diagnose debug reset
diagnose debug application ike -1
diagnose debug enable
```

With PFS enabled, each phase 2 rekey carries a new key exchange payload

► 10.3 PFS in the Phase 2 Configuration

Edit Phase 2

Name: to-Site2

Comments: VPN: to-Site2 (Created by VPN wizard)

Local Address: Named Address ▼ to-Site2_local ▼

Remote Address: Named Address ▼ to-Site2_remote ▼

Advanced...

Phase 2 Proposal: Add

Encryption: DES ▼ Authentication: SHA256 ▼

Enable Replay Detection: ☒

Enable Perfect Forward Secrecy (PFS): ☒

Diffie-Hellman Group:

☐ 32	☐ 31	☐ 30	☐ 29	☐ 28	☐ 27
☐ 21	☐ 20	☐ 19	☐ 18	☐ 17	☐ 16
☐ 15	☒ 14	☐ 5	☐ 2	☐ 1	

Figure 32 — SITE1 phase 2 — PFS enabled, DH group 14

Edit Phase 2

Name: to-Site1

Comments: VPN: to-Site1 (Created by VPN wizard)

Local Address: Named Address ▼ to-Site1_local ▼

Remote Address: Named Address ▼ to-Site1_remote ▼

Advanced...

Phase 2 Proposal: Add

Encryption: DES ▼ Authentication: SHA256 ▼

Enable Replay Detection: ☒

Enable Perfect Forward Secrecy (PFS): ☒

Diffie-Hellman Group:

☐ 32	☐ 31	☐ 30	☐ 29	☐ 28	☐ 27
☐ 21	☐ 20	☐ 19	☐ 18	☐ 17	☐ 16
☐ 15	☒ 14	☐ 5	☐ 2	☐ 1	

Figure 33 — SITE2 phase 2 — matching configuration

Replay detection is also enabled, which is the anti-replay protection listed in section 1.1 — the mechanism that causes a duplicated packet to be discarded rather than accepted a second time.

11. Test 3 — IKEv1 Compared with IKEv2

The two IKE versions establish the same tunnel with different numbers of round trips. Capturing each negotiation and counting packets makes the difference concrete.

► 11.1 Method

```
diagnose sniffer packet port3 "port 500 or port 4500" 4      # session 1
diagnose vpn ike gateway clear                               # session 2
```

```
Site1 # diagnose sniffer packet port3 "port 500 or port 4500" 4
Using Original Sniffing Mode
interfaces=[port3]
filters=[port 500 or port 4500]
```

Figure 34 — Capture running in the first CLI session

```
CLI Console (3)
Site1 # diagnose vpn ike gateway clear
Site1 #
```

Figure 35 — Negotiation forced from the second session

Two sessions are needed because the capture blocks its own console. Clearing the gateway from a second session forces a complete re-negotiation, so the whole exchange is captured rather than joined mid-session.

► 11.2 IKEv1

```
Site1 # diagnose sniffer packet port3 "port 500 or port 4500" 4
Using Original Sniffing Mode
interfaces=[port3]
filters=[port 500 or port 4500]
14.215710 port3 -- 10.255.255.1.500 -> 10.255.255.2.500: udp 84
14.216157 port3 -- 10.255.255.1.500 -> 10.255.255.2.500: udp 100
14.326213 port3 -- 10.255.255.1.500 -> 10.255.255.2.500: udp 324
14.327072 port3 -- 10.255.255.2.500 -> 10.255.255.1.500: udp 188
14.327450 port3 -- 10.255.255.1.500 -> 10.255.255.2.500: udp 390
14.329050 port3 -- 10.255.255.2.500 -> 10.255.255.1.500: udp 390
14.330050 port3 -- 10.255.255.1.500 -> 10.255.255.2.500: udp 108
14.330576 port3 -- 10.255.255.2.500 -> 10.255.255.1.500: udp 84
14.331500 port3 -- 10.255.255.1.500 -> 10.255.255.2.500: udp 460
14.333200 port3 -- 10.255.255.2.500 -> 10.255.255.1.500: udp 436
14.333800 port3 -- 10.255.255.1.500 -> 10.255.255.2.500: udp 60
^C
11 packets received by filter
0 packets dropped by kernel
```

Figure 36 — IKEv1 negotiation — 11 packets captured

Packets	Exchange	Purpose
1–2	Informational	Delete notifications tearing down the previous SA
3–4	Main mode 1–2	Security association proposals and selection
5–6	Main mode 3–4	Diffie-Hellman key exchange and nonces
7–8	Main mode 5–6	Identity and authentication, now encrypted

Packets	Exchange	Purpose
9–11	Quick mode	Phase 2 security associations

Nine packets establish the tunnel; the first two are teardown of the previous session. **Packet sizes tell the same story — the 380-byte messages carry the Diffie-Hellman material, dropping to around 100 bytes once encryption is active.**

► 11.3 Switching to IKEv2

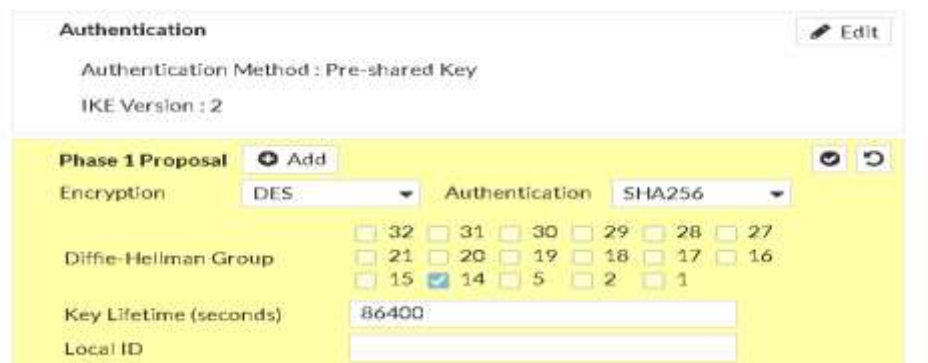


Figure 37 — IKE version set to 2, phase 1 proposal unchanged



Figure 38 — Tunnel re-established on IKEv2

Both peers must be changed together. A version mismatch produces no tunnel at all, and the debug output names it only if IKE debugging is already running.

► 11.4 IKEv2

```
Site1 # diagnose sniffer packet port3 "port 500 or port 4500" 4
Using Original Sniffing Mode
interfaces=[port3]
filters=[port 500 or port 4500]
10.766130 port3 -- 10.255.255.1.500 -> 10.255.255.2.500: udp 72
10.768058 port3 -- 10.255.255.2.500 -> 10.255.255.1.500: udp 64
11.766308 port3 -- 10.255.255.1.500 -> 10.255.255.2.500: udp 436
11.769187 port3 -- 10.255.255.2.500 -> 10.255.255.1.500: udp 420
11.770260 port3 -- 10.255.255.1.500 -> 10.255.255.2.500: udp 216
11.771617 port3 -- 10.255.255.2.500 -> 10.255.255.1.500: udp 208
^C
6 packets received by filter
0 packets dropped by kernel
```

Figure 39 — IKEv2 negotiation — 6 packets captured

Packets	Exchange	Purpose
1–2	Informational	Delete notifications for the previous SA
3–4	IKE_SA_INIT	Proposals, Diffie-Hellman exchange and nonces in a single round trip

Packets	Exchange	Purpose
5–6	IKE_AUTH	Identity, authentication and the first child SA, encrypted

► 11.5 The Comparison

	IKEv1	IKEv2
Packets to establish	9	4
Round trips	4.5	2
Phase 2 negotiation	Separate quick mode exchange	Carried inside IKE_AUTH
Dead peer detection	Bolt-on extension (RFC 3706)	Built in as liveness checks
Aggressive mode	Present — exposes a PSK hash	No equivalent
Roaming support	None	MOBIKE

Why the reduction matters: fewer round trips means faster establishment on high-latency links, and **noticeably faster convergence at scale — a hub with hundreds of spokes re-establishing after an outage.**

12. Cipher Constraint

The evaluation licence on the FortiGate VM restricts the available encryption algorithms.

```
CLI Console (1)
Site1 # config vpn ipsec phase1-interface
Site1 (phase1-interface) # edit "to-Site2"
Site1 (to-Site2) # set proposal
des-md5          des-md5
des-sha1          des-sha1
des-sha256        des-sha256
des-sha384        des-sha384
des-sha512        des-sha512
```

Figure 40 — The CLI proposal list — only DES variants are offered

Only DES is available, so the tunnel negotiates 56-bit encryption. DES is considered broken and a production deployment would use AES-256 with SHA-256, or AES-256-GCM.

What was tightened within the constraint: SHA-256 selected in place of MD5 for integrity, Diffie-Hellman group 14 only, PFS enabled, and replay detection on.

The constraint does not affect what this lab demonstrates. Negotiation, SA establishment, ESP encapsulation, DPD behaviour and PFS rekeying are all independent of the cipher chosen.

13. Summary

Component	Result
Tunnel type	Route-based site-to-site IPsec, tunnel mode
Transit	10.255.255.0/30 isolated segment, no NAT between sites
Protected subnets	192.168.10.0/24 ↔ 192.168.20.0/24
Wizard output	Phase 1 and 2, address groups, static route, blackhole route, both policies
DPD	Disabled — SA stayed up with a dead peer. On Idle — detected within seconds
PFS	Enabled with DH group 14; rekey observed at the shortened 300-second lifetime
IKE versions	IKEv1 9 packets, IKEv2 4 packets; final configuration on IKEv2
Cipher	DES — evaluation licence limitation, documented rather than worked around